

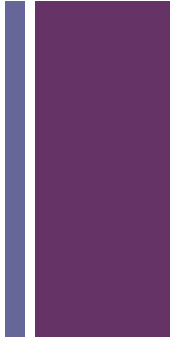
# + Chương 7

Thiết bị ngoại vi





# Chương 7. Thiết bị ngoại vi



7.1 Các thiết bị ngoại vi

7.2 Module vào/ra

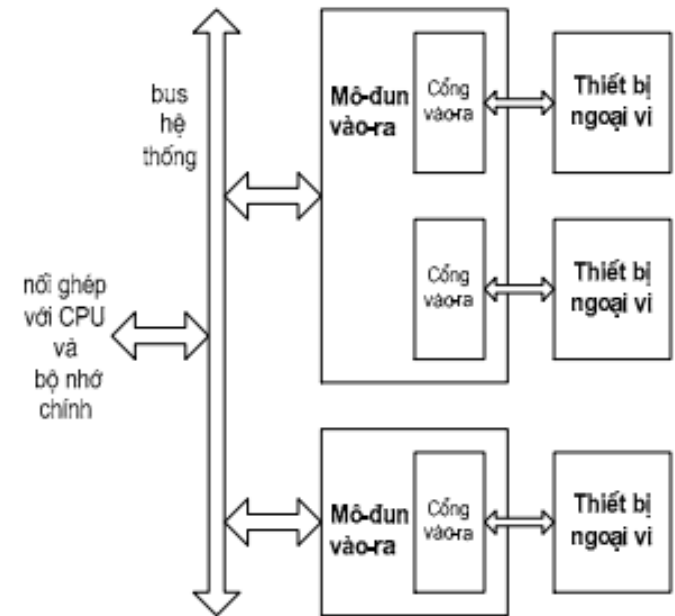
7.3 Các kỹ thuật I/O

- a. I/O chương trình
- b. Điều khiển ngắt vào/ra
- c. Truy xuất bộ nhớ trực tiếp

7.4 Các bộ xử lý và kênh vào/ra

# + 7.1 Thiết bị ngoại vi

- Một trong ba thành phần cơ bản của hệ thống máy tính: CPU, bộ nhớ và thiết bị ngoại vi (thông qua module I/O)
- Chức năng: trao đổi dữ liệu giữa máy tính với bên ngoài
- Kết nối với máy tính qua module vào/ra (module I/O)
  - Module I/O: Truyền các thông tin điều khiển, dữ liệu và địa chỉ giữa CPU và thiết bị ngoại vi
- Có ba loại
  - Con người đọc được: màn hình, máy in,...
  - Máy đọc được: ổ cứng, cảm biến, băng từ,...
  - Truyền thông: modem, card mạng,...





# Mô hình tổng quát của I/O Module

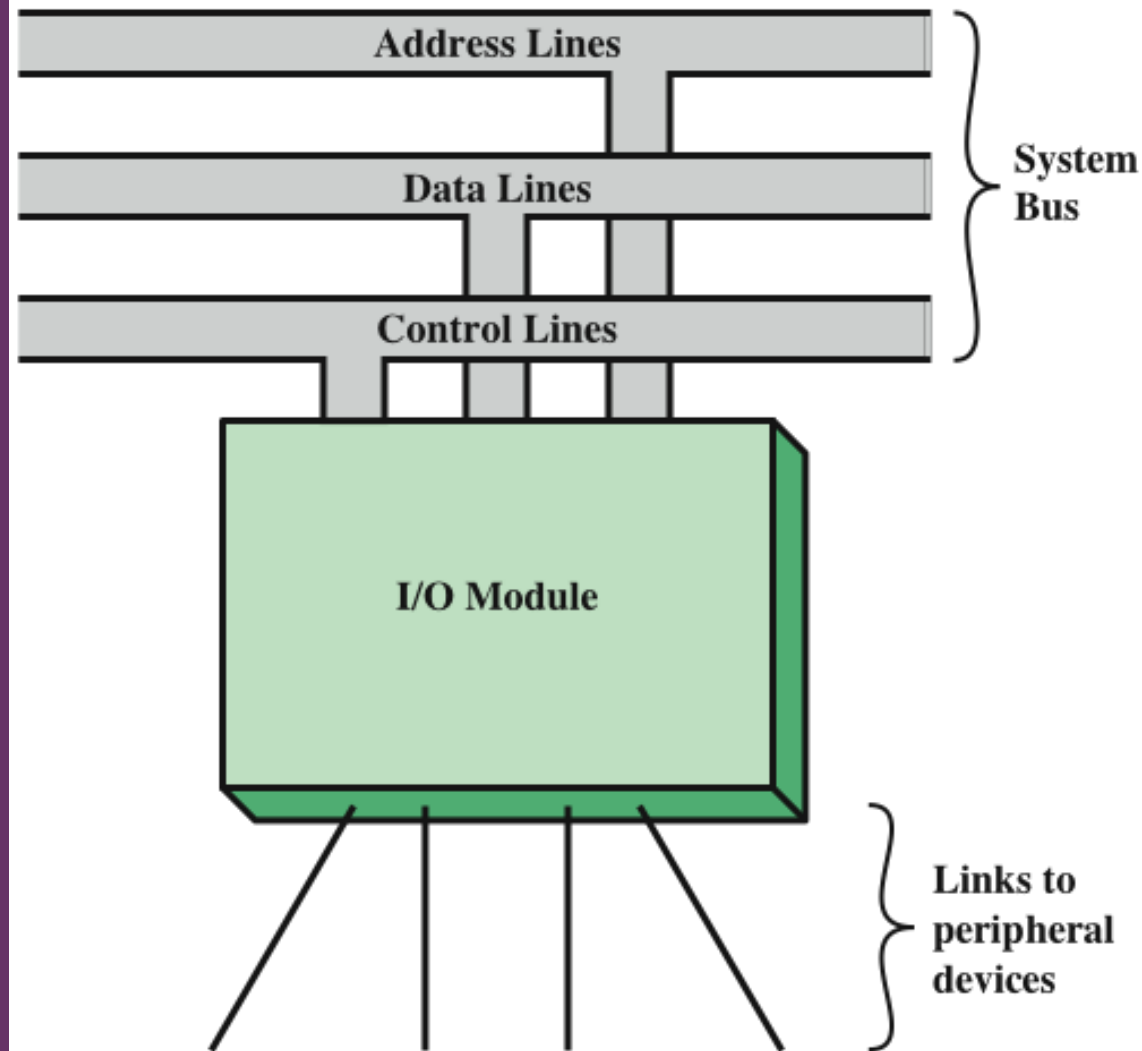
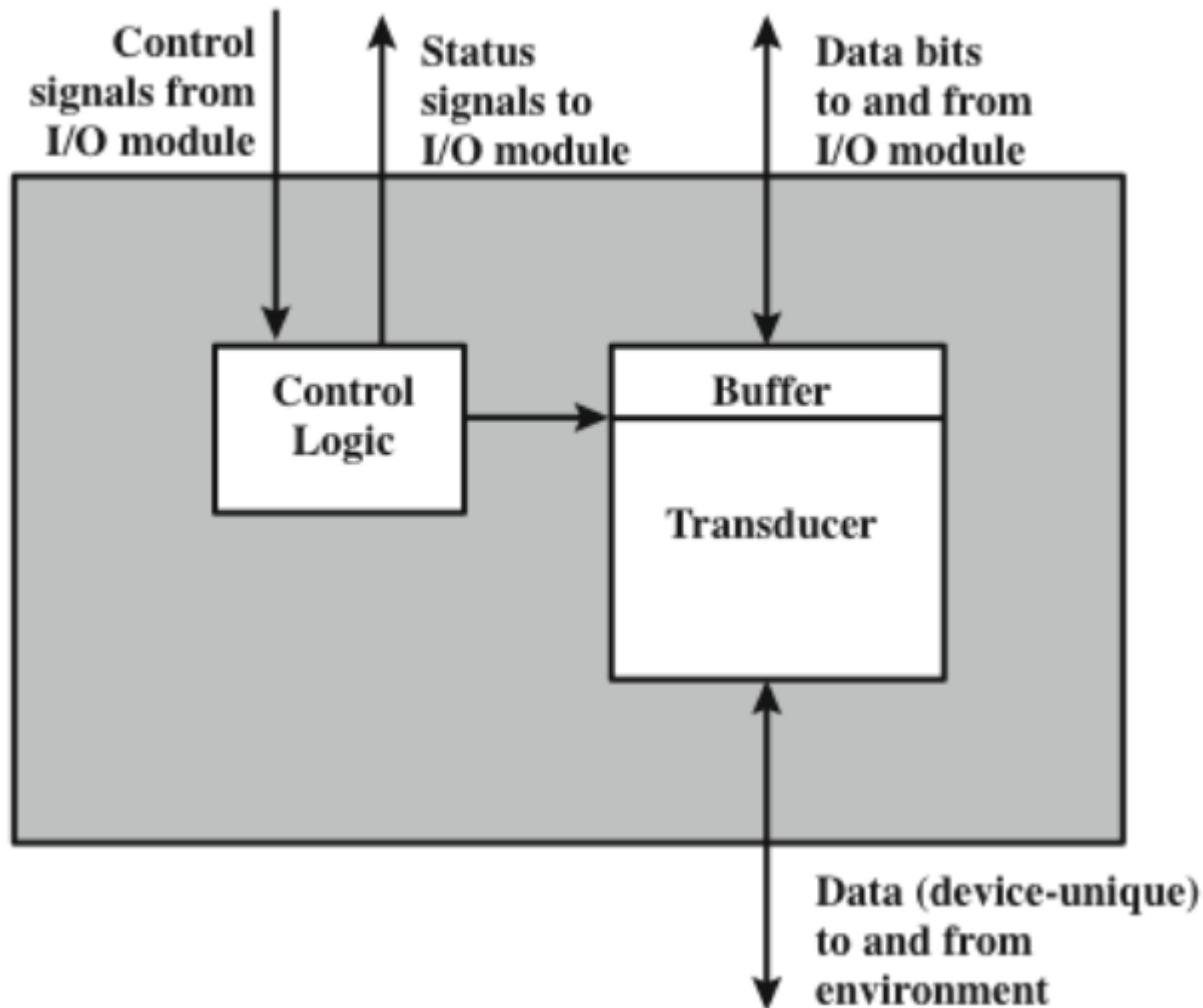


Figure 7.1 Generic Model of an I/O Module

## + a. Sơ đồ khối thiết bị ngoại vi



# + a. Sơ đồ khối thiết bị ngoài (tiếp)



## ■ Giao diện với module I/O:

- *Tín hiệu điều khiển - Control signal*: xác định chức năng mà thiết bị sẽ thực hiện:
  - READ: yêu cầu thiết bị gửi dữ liệu vào module I/O (INPUT)
  - WRITE: yêu cầu thiết bị nhận dữ liệu từ module I/O (OUTPUT)
  - Các tín hiệu điều khiển đặc biệt
- *Dữ liệu – Data*: một tập các bit được gửi đến hoặc đi từ module I/O.
- *Tín hiệu trạng thái - Status signal*: cho biết trạng thái của thiết bị. Ví dụ:
  - READY: thiết bị sẵn sàng cho việc truyền dữ liệu.
  - NOT-READY: không sẵn sàng truyền dữ liệu

## ■ Logic điều khiển – Control logic: nhận các tín hiệu điều khiển từ module I/O, điều khiển hoạt động của thiết bị.

## ■ Bộ chuyển đổi - Transducer: chuyển đổi dữ liệu (đang ở dạng t/h điện) sang các dạng khác (vd: điểm ảnh trên màn hình,...) và ngược lại.

## ■ Bộ đệm (buffer) để lưu trữ tạm dữ liệu đang được chuyển giao giữa module I/O và môi trường bên ngoài; kích thước bộ đệm thường từ 8 đến 16 bit.

+

## b. Bàn phím/Màn hình

### Bảng chữ cái tham khảo quốc tế (IRA)

- Ký tự
  - Gắn với mỗi ký tự là một mã
  - Mỗi ký tự được biểu diễn bởi một mã nhị phân 7-bit : biểu diễn 128 ký tự
- Hai loại ký tự:
  - In được
    - Các ký tự chữ cái, số và ký tự đặc biệt có thể được in trên giấy hoặc hiển thị trên màn hình
  - Điều khiển
    - Điều khiển việc in/hiển thị các ký tự
    - VD: carriage return
    - Các ký tự điều khiển khác liên quan đến các thủ tục truyền tin

Công cụ tương tác máy tính/  
người dùng phổ biến nhất  
Người dùng cung cấp đầu vào  
thông qua bàn phím  
Màn hình hiển thị dữ liệu được  
cung cấp bởi máy tính  
Đơn vị chuyển đổi cơ bản là ký  
tự

### Mã bàn phím

- Khi người dùng bấm một phím, một tín hiệu điện tử được tạo ra bởi một bộ chuyển đổi trong bàn phím và dịch sang mẫu bit của mã IRA tương ứng
- Mẫu bit này được truyền đến mô-đun I/O trong máy tính
- Trên đầu ra, các ký tự mã IRA được truyền đến một thiết bị ngoại vi từ module I/O
- Bộ chuyển đổi giải mã và gửi các tín hiệu điện tử yêu cầu đến thiết bị đầu ra để hiển thị ký tự được chỉ định hoặc thực hiện chức năng điều khiển yêu

## 7.2 Module I/O



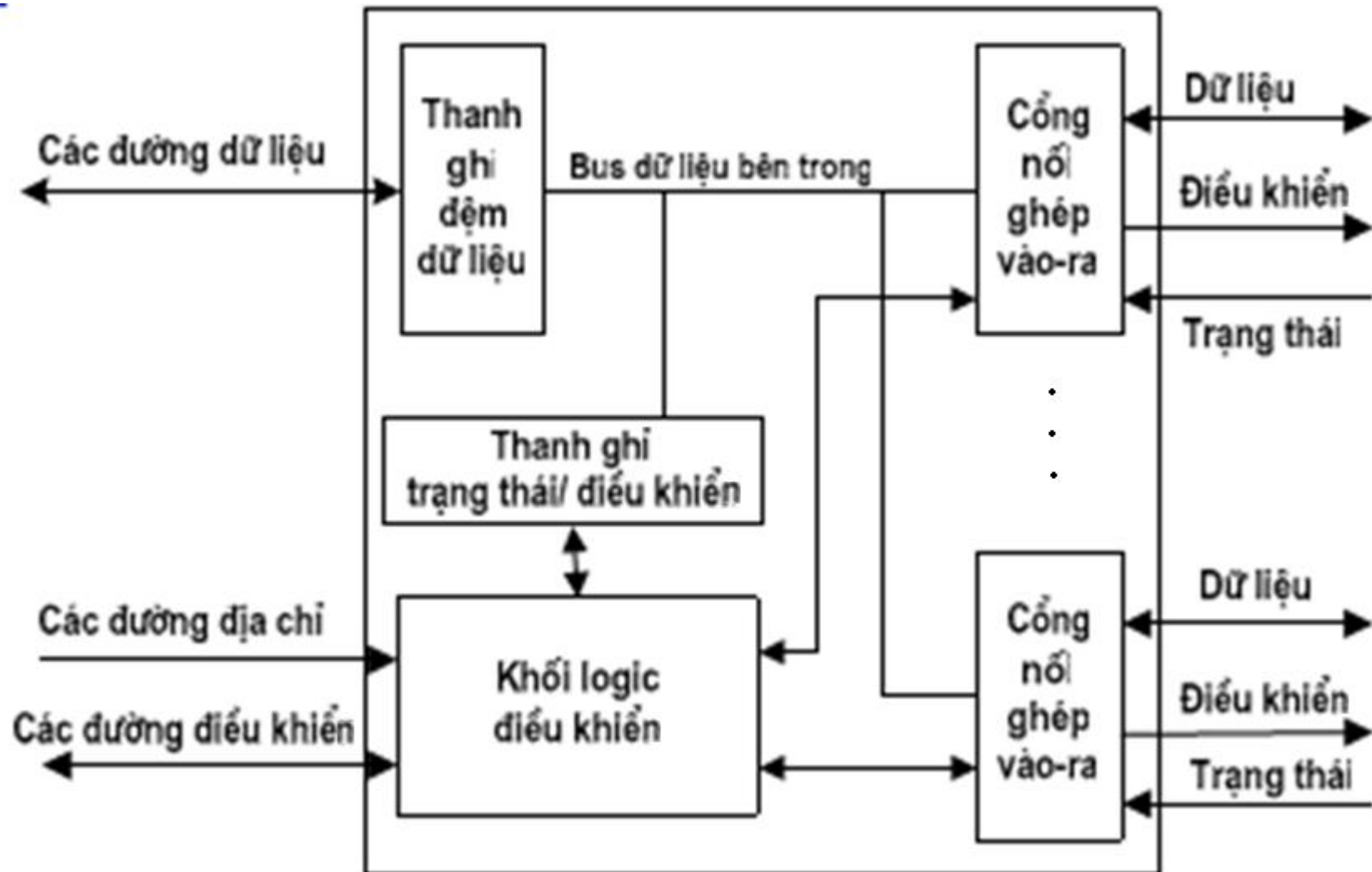
### a. Chức năng

Các chức năng chính của một module I/O gồm:

- **Điều khiển và định thời:** phối hợp luồng lưu lượng truy cập giữa thành phần thiết bị bên trong (main memory, bus) và thiết bị ngoại vi
- **Trao đổi thông tin với VXL:** gồm giải mã lệnh, dữ liệu, báo cáo trạng thái (trạng thái của thiết bị I/O có sẵn sàng hay không), nhận dạng địa chỉ (địa chỉ các cổng mà TBNV được nối vào)
- **Trao đổi thông tin với TBNV:** gồm các lệnh, thông tin trạng thái và dữ liệu
- **Đệm dữ liệu:** thực hiện các hoạt động đệm cần thiết để cân bằng tốc độ TBNV và bộ nhớ
- **Phát hiện và báo cáo lỗi**



## b. Cấu trúc Module I/O



## + b. Cấu trúc Module I/O (tiếp)

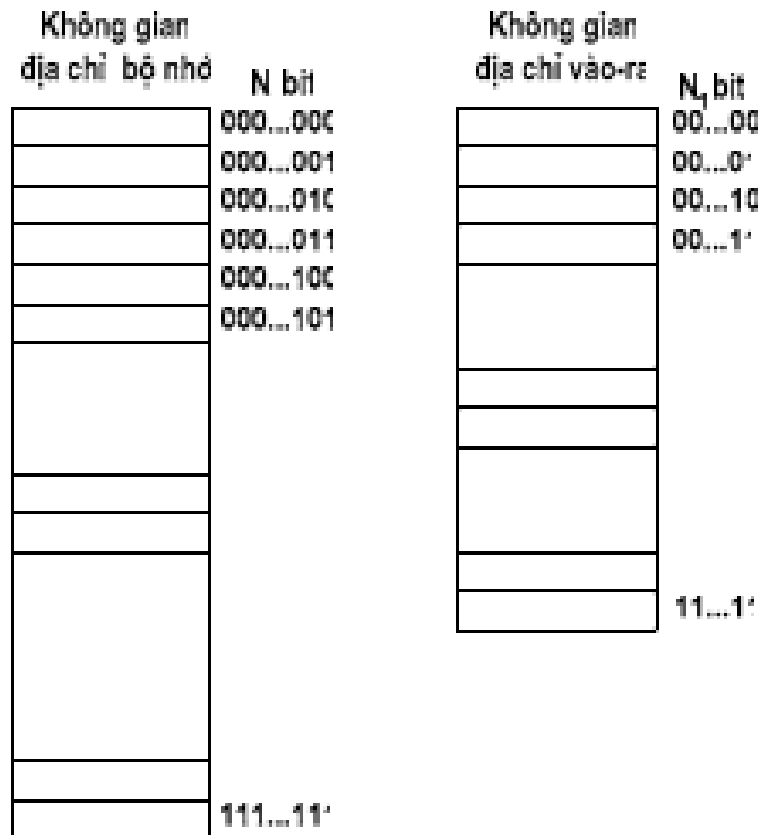


Các module I/O thay đổi khác nhau theo sự phức tạp và số lượng các thiết bị ngoài mà nó điều khiển. Cấu trúc chung nhất:

- Dữ liệu được truyền đến và đi từ module được đệm qua một hoặc nhiều **thanh ghi dữ liệu (data register)**.
- **Thanh ghi trạng thái/ điều khiển (status/control register)**: lưu trữ thông tin trạng thái của thiết bị hoặc thông tin điều khiển của bộ VXL
- **Khối logic điều khiển - I/O logic**: tương tác với VXL qua một tập các **đường điều khiển (control line)**. VXL sử dụng các đường điều khiển để ra lệnh cho module I/O. Module I/O cũng có thể sử dụng một số đường điều khiển để gửi các tín hiệu phân xử bus hoặc tín hiệu trạng thái.
- Module cũng có khả năng nhận diện và sinh ra các địa chỉ với mỗi thiết bị được nối đến nó (địa chỉ cổng). Mỗi module I/O có một (nếu chỉ nối với một TBNV) hoặc một tập địa chỉ (nếu module nối với nhiều TBNV)
- **Cổng nối ghép vào/ra (External Device Interface Logic)**: giao tiếp với thiết bị ngoại vi

## + c. Địa chỉ cổng vào/ra

- Cũng giống như bộ nhớ, các TBNV được gắn vào module I/O qua các cổng. Để CPU giao tiếp được với các TBNV, các cổng này phải được gán một giá trị địa chỉ



# + Không gian địa chỉ I/O

- Có hai phương thức thực hiện không gian địa chỉ cho các TBNV:

- ***I/O ánh xạ bộ nhớ (memory-mapped I/O):***

- Bộ nhớ và TBNV chia sẻ chung không gian địa chỉ. VXL coi các thanh ghi dữ liệu và trạng thái như các ô nhớ và sử dụng cùng các lệnh để truy cập cả bộ nhớ và thiết bị ngoại vi
- Chỉ sử dụng một đường đọc và ghi, do đó bus phải sắp xếp giữa việc đọc/ghi bộ nhớ và vào/ra TBNV

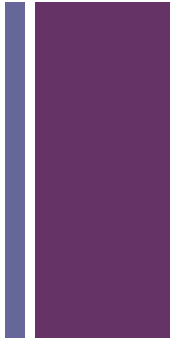
- ***I/O riêng biệt (isolated I/O):***

- Sử dụng một đường command line để xác định: địa chỉ BN hay địa chỉ TBNV
- Toàn bộ dải địa chỉ dùng cho cả hai. VD: 10 đường địa chỉ cho phép đánh địa chỉ 1024 ô nhớ và 1024 TBNV
- Tập các chỉ lệnh đến BN và TBNV khác nhau



## Ví dụ

- Bộ xử lý 680x0 của Motorola : quản lý một không gian địa chỉ chung cho cả bộ nhớ và I/O.
- Bộ xử lý Intel Pentium:
  - Không gian địa chỉ bộ nhớ =  $2^{32}$  địa chỉ
  - Không gian địa chỉ vào-ra =  $2^{16}$  địa chỉ
  - Tín hiệu điều khiển: M/IO
  - Lệnh vào-ra chuyên dụng: IN, OUT



## + 7.3. Các kỹ thuật vào/ra

Hoạt động của module I/O theo 3 kỹ thuật sau:

### ■ I/O chương trình

- CPU thực thi một chương trình trực tiếp điều khiển các hoạt động vào/ra
- Khi bộ xử lý ra lệnh, nó phải đợi cho đến khi hoạt động vào/ra hoàn tất
- Bộ xử lý chạy nhanh hơn module I/O sẽ gây lãng phí thời gian xử lý

### ■ I/O điều khiển ngắt

- Bộ vi xử lý ra lệnh I/O sau đó tiếp tục thi hành các lệnh tiếp theo trong chương trình.
- Khi module I/O hoàn thành công việc, nó sẽ gửi tín hiệu yêu cầu ngắt đến VXL.

### ■ Truy cập bộ nhớ trực tiếp (DMA)

- Module I/O và bộ nhớ chính trực tiếp trao đổi dữ liệu mà không có sự tham gia của bộ vi xử lý

# Các kỹ thuật I/O

+

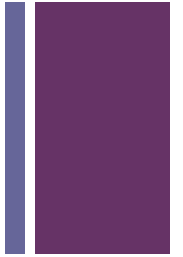
|                                          | No Interrupts  | Use of Interrupts          |
|------------------------------------------|----------------|----------------------------|
| I/O-to-memory transfer through processor | Programmed I/O | Interrupt-driven I/O       |
| Direct I/O-to-memory transfer            |                | Direct memory access (DMA) |

## + a. Kỹ thuật I/O chương trình

- Khi cần thực hiện một tác vụ vào/ra:
  - VXL thực thi một chương trình và gửi lệnh đến module I/O tương ứng
  - Module I/O nhận yêu cầu, thiết lập các bit trạng thái trên thanh ghi trạng thái
  - CPU định kỳ kiểm tra trạng thái của module I/O
    - Chưa sẵn sàng thì tiếp tục định kỳ kiểm tra
    - Đã sẵn sàng, thiết lập việc truyền dl đến module I/O



## + a. Các lệnh I/O – I/O command (từ VXL đến module I/O)



Để thực thi một lệnh vào/ra, VXL thực hiện công việc sau:

- **Đặt địa chỉ lên bus địa chỉ:** định ra module I/O và TBNV cụ thể
- **Đưa các mệnh lệnh vào/ra:** Thiết lập các đường điều khiển trong bus điều khiển. Có 4 loại mệnh lệnh vào/ra:

**1) Control:** kích hoạt một thiết bị ngoại vi và chỉ định nó phải làm gì

**2) Test:** kiểm tra các điều kiện trạng thái liên quan đến một module I/O và các thiết bị ngoại vi: *TBNV bật hay tắt, hoạt động I/O đang thực hiện đã xong chưa, có lỗi gì*

**3) Read:** yêu cầu đọc dữ liệu từ TBNV vào VXL

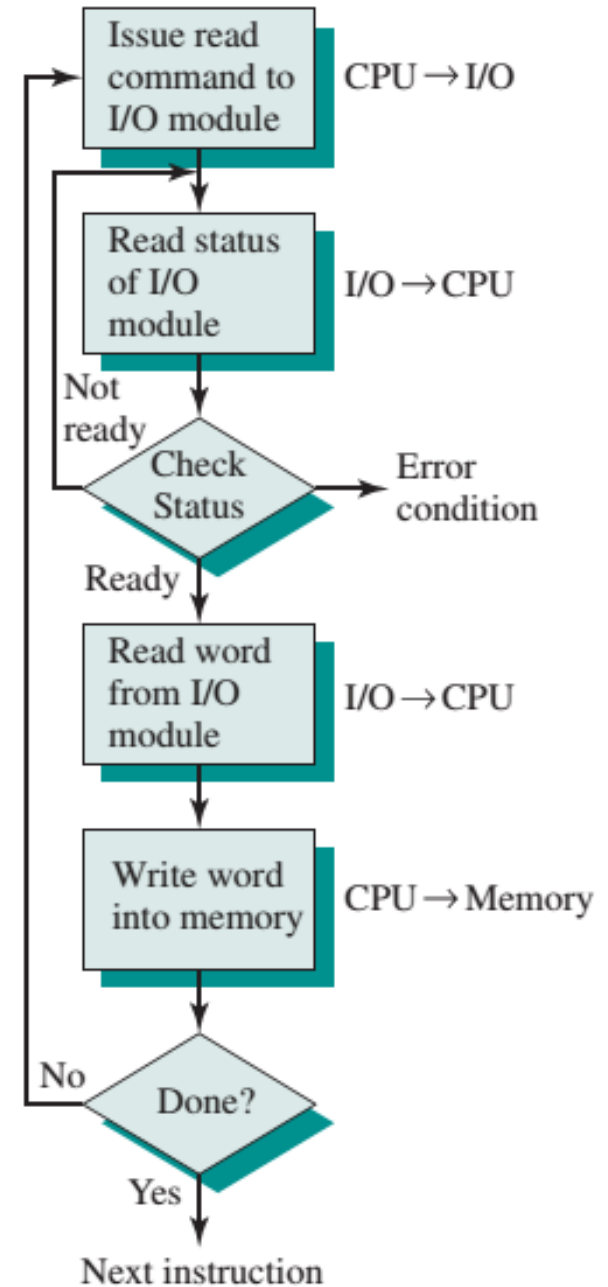
- Module I/O lấy dữ liệu từ thiết bị ngoại vi và đặt nó vào bộ đệm bên trong → đặt dữ liệu vào bus cho CPU

**4) Write:** yêu cầu ghi dữ liệu ra TBNV

- Module I/O lấy dữ liệu từ bus dữ liệu rồi chuyển dữ liệu đó đến thiết bị ngoại vi

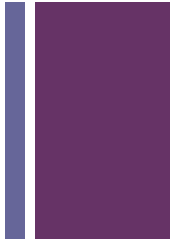


Ví dụ: Hoạt  
động đọc  
(READ) từ thiết  
bị ngoại vi vào  
RAM



(a) Programmed I/O

## + b. I/O điều khiển ngắt



- Vấn đề với **I/O chương trình** là bộ xử lý phải đợi một thời gian dài để module I/O sẵn sàng cho việc nhận hoặc truyền dữ liệu
- Giải pháp thay thế là bộ vi xử lý ra lệnh I/O cho module, sau đó thực hiện các việc khác.
- Khi module I/O sẵn sàng trao đổi dữ liệu với VXL, nó sẽ gửi tín hiệu ngắt đến VXL
- Bộ xử lý thực hiện việc truyền dữ liệu và tiếp tục quá trình xử lý trước đó



# Cơ chế làm việc

## Từ phía VXL

- VXL đưa ra lệnh READ.
- Sau đó thực hiện các công việc khác (vd: trong trường hợp có nhiều CT đang chạy tại một thời điểm)
- Sau mỗi chu kỳ lệnh, VXL sẽ kiểm tra xem có tín hiệu yêu cầu ngắt được gửi tới
- Nếu có, VXL lưu trữ nội dung đang thực hiện và xử lý ngắt
- VXL nhận dữ liệu từ bus lưu trữ vào bộ nhớ và tiếp tục chương trình

## Từ phía module I/O

- Nhận lệnh READ từ VXL
- Đọc dữ liệu vào từ TBNV tương ứng
- Khi dữ liệu được đưa vào thanh ghi dữ liệu, module gửi tín hiệu yêu cầu ngắt đến VXL và chờ đợi tín hiệu yêu cầu dữ liệu từ VXL.
- Khi có tín hiệu đó, module đặt dữ liệu vào bus và sẵn sàng để thực hiện các hoạt động I/O khác

## Phần cứng

TBNV đưa ra ngắt

Bộ xử lý kết thúc  
lệnh hiện tại

Bộ xử lý gửi  
xác nhận việc  
nhận được ngắt

Bộ xử lý đưa  
dữ liệu PSW và PC  
vào ngăn xếp

Bộ xử lý tải  
giá trị mới vào PC tùy  
theo ngắt được yêu cầu

## Phần mềm

Lưu các thông tin  
trạng thái

Xử lý ngắt

Khôi phục lại  
thông tin trạng thái

Khôi phục lại  
PSW và PC





# Cơ chế xử lý ngắt

## VXL nhận được yêu cầu ngắt

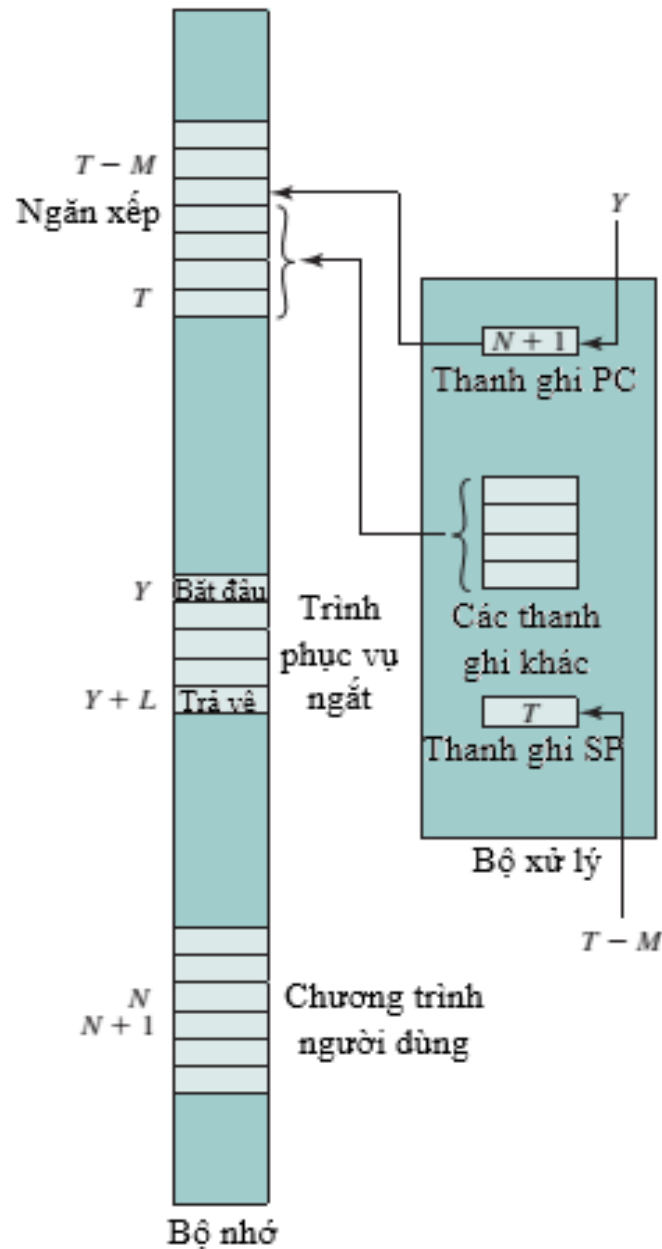
- Thiết bị phát tín hiệu ngắt cho bộ xử lý
- Bộ xử lý hoàn thành lệnh hiện tại → Kiểm tra thấy có y/c ngắt → gửi ACK báo đã nhận ngắt.
- Chuyển sang chế độ phục vụ ngắt: lưu trữ nội dung các thanh ghi vào vùng ngăn xếp của RAM (hình trang sau)
- Tải trình điều khiển ngắt: đặt địa chỉ đầu tiên của trình này vào thanh ghi PC → thực hiện hoạt động vào/ra

## VXL thực hiện xong yêu cầu ngắt

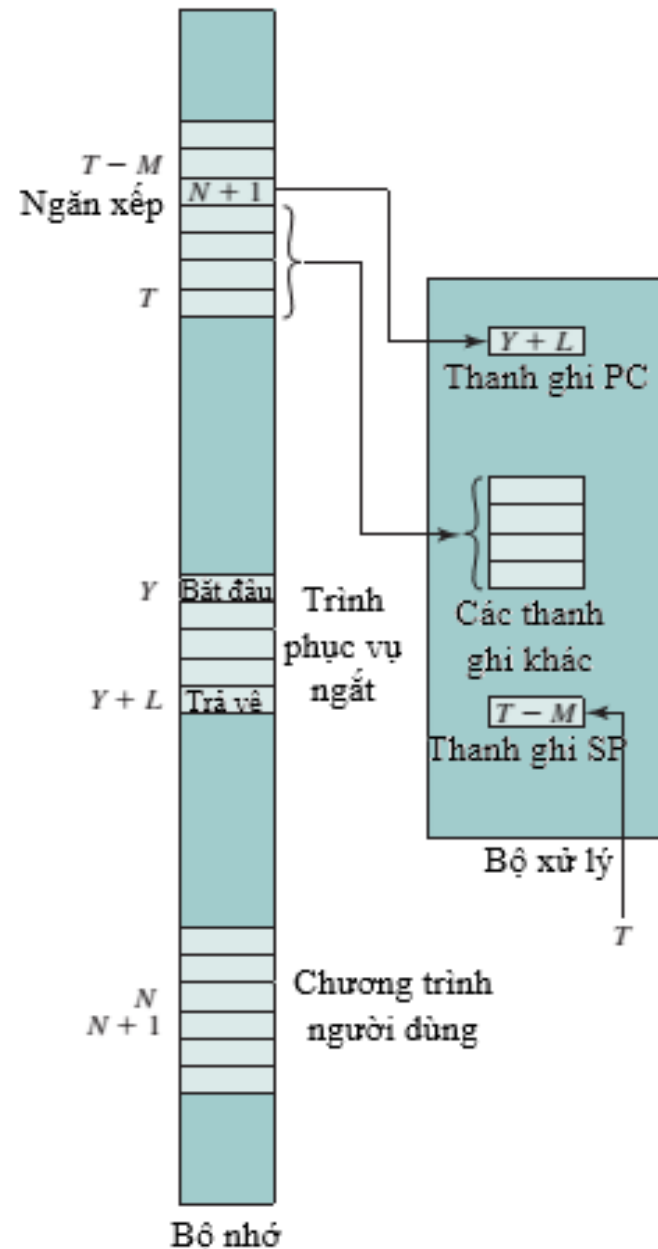
- Sau khi thực hiện xong hoạt động vào/ra, CPU khôi phục lại công việc đang thực hiện
- Nạp lại nội dung từ vùng ngăn xếp vào các thanh ghi: PSW, PC
- Khôi phục lại luồng điều khiển



- a. Quá trình chuyển sang chế độ phục vụ ngắt
- b. Khôi phục lại luồng điều khiển sau khi thực hiện yêu cầu ngắt xong



a. Lưu chương trình hiện tại trước khi xử lý ngắt



b. Khôi phục lại chương trình

# Hai vấn đề phát sinh

Hai vấn đề thiết kế phát sinh khi thực hiện I/O điều khiển ngắt:

1. **Nhận diện thiết bị:** Bởi vì sẽ có nhiều module I/O, khi có một yêu cầu ngắt gửi tới, bộ vi xử lý sẽ xác định thiết bị đưa ra yêu cầu ngắt bằng cách nào?
2. **Xác định ưu tiên.** Nếu xảy ra nhiều ngắt cùng một thời điểm, VXL lựa chọn ngắt nào để xử lý?

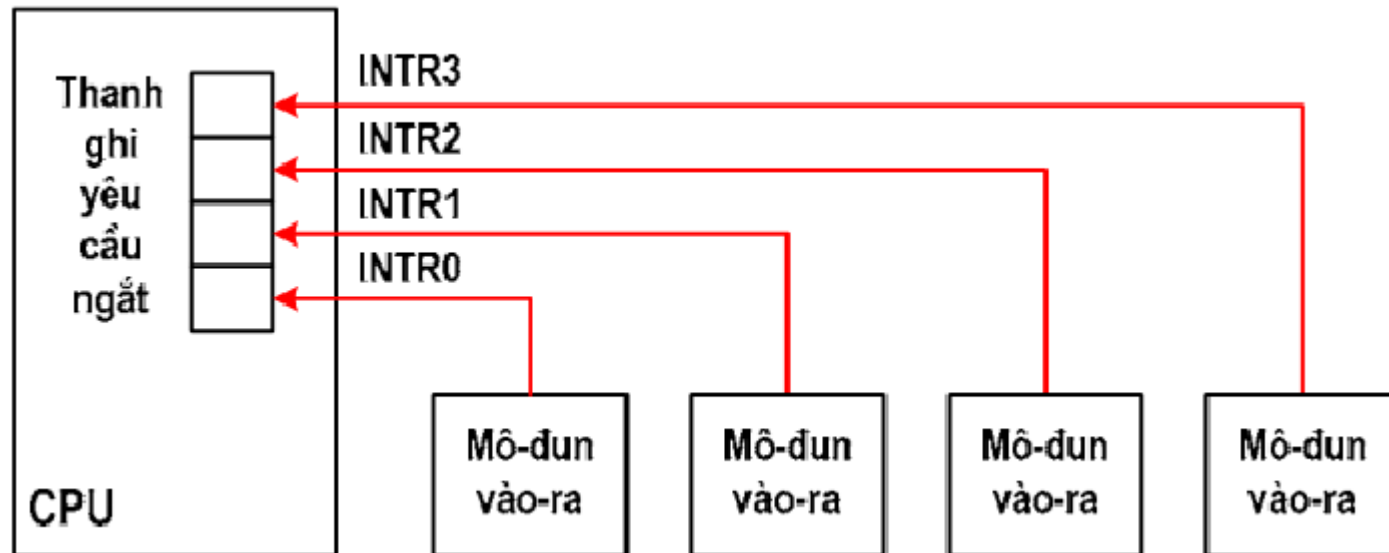


# + 1. Nhận diện thiết bị

Bốn loại kỹ thuật chung được sử dụng phổ biến:

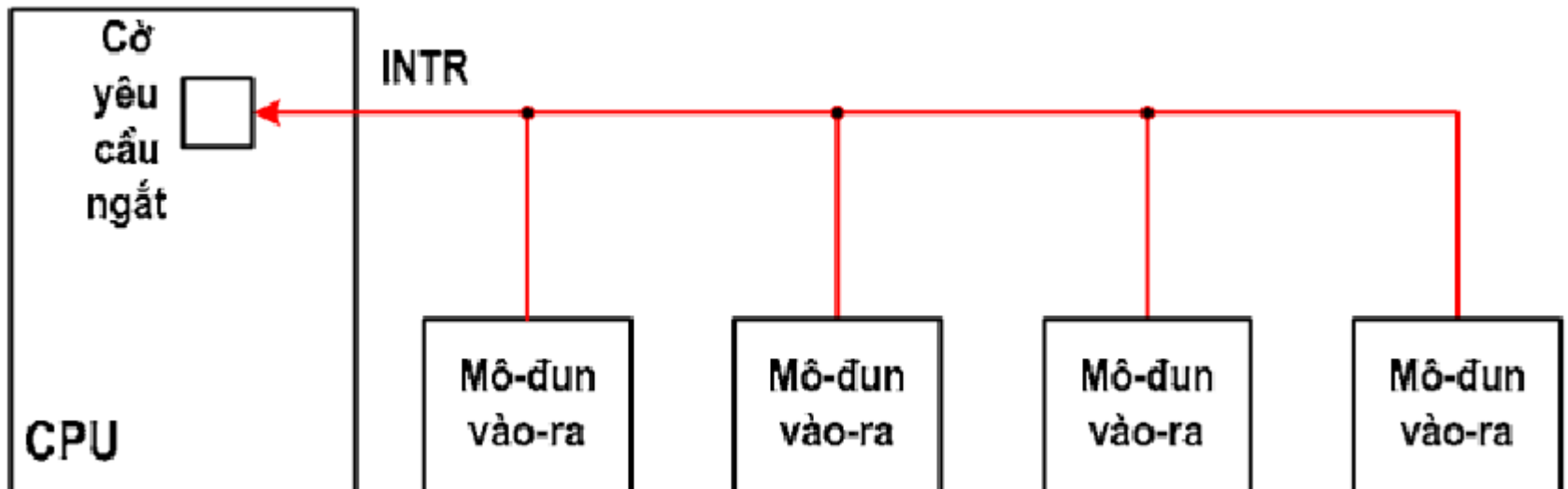
## ■ Nhiều đường ngắt

- Sử dụng nhiều đường ngắt giữa VXL và các module I/O → dễ dàng xác định thiết bị
- Không thực tế do kỹ thuật này làm tăng số đường bus và các chân của VXL. Thêm vào đó, vẫn phải có nhiều module I/O nối với một đường → vẫn cần một trong ba kỹ thuật còn lại



## + ■ Thăm dò phần mềm

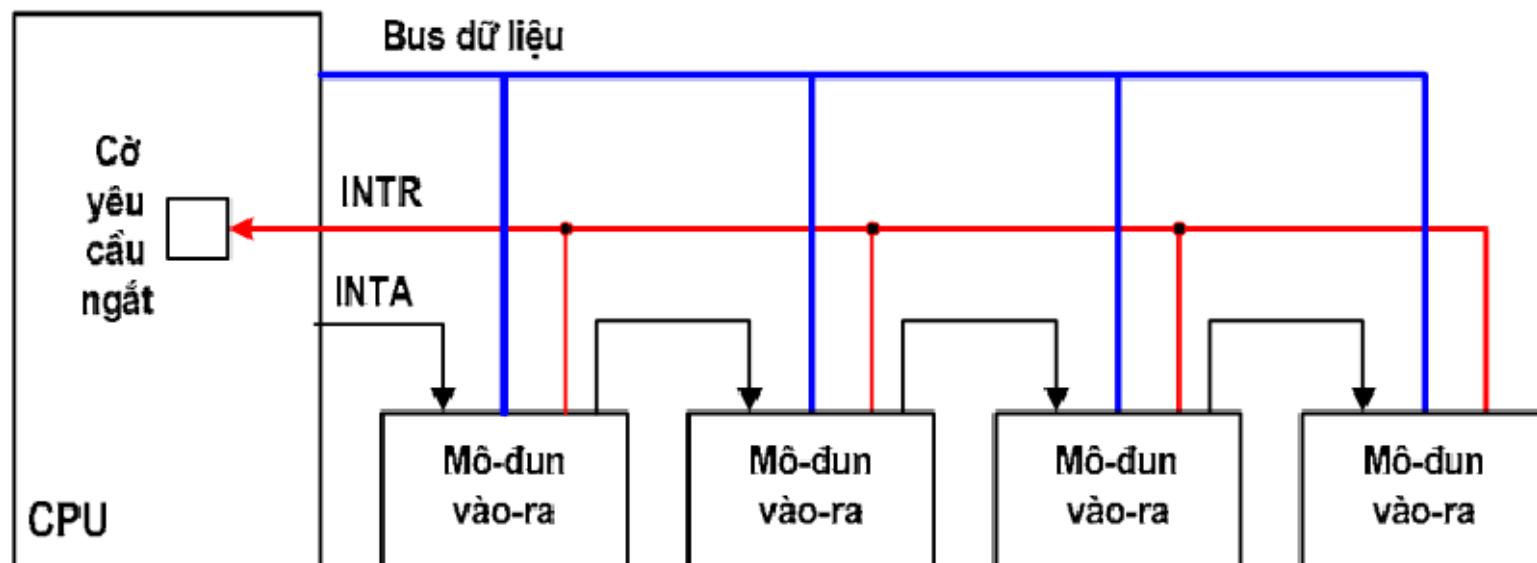
- Khi bộ xử lý phát hiện ra một ngắt, nó thực thi một phần mềm thăm dò:
  - Thực hiện lệnh thăm dò (vd: lệnh TEST I/O) tới từng module I/O (thông qua địa chỉ) → Module I/O sẽ trả lời nếu nó đưa ra y/c ngắt.
  - Hoặc thực hiện lệnh đọc thanh ghi trạng thái của từng module để phát hiện module y/c ngắt
- Nhược điểm: Tốn thời gian



## ■ Chuỗi Daisy (thăm dò phần cứng, vector)



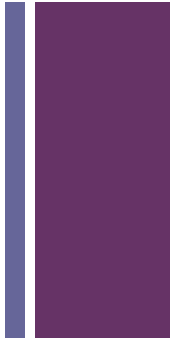
- Tất cả các module I/O sử dụng chung một đường yêu cầu ngắt (INTR)
- Đường nhận biết ngắt (INTA) được nối chuỗi qua các module
- Khi VXL nhận được y/c ngắt, nó sẽ gửi lại một tín hiệu ACK qua đường INTA
- T/h này truyền qua các module I/O đến khi gặp module y/c ngắt. Module này trả lời bằng cách đặt một word lên bus dữ liệu: được gọi là **vector ngắt** (chứa thông tin địa chỉ của module I/O hoặc mã nhận dạng thiết bị khác).
- VXL sử dụng vector này trở tới trình phục vụ ngắt tương ứng của thiết bị → ngắt vector





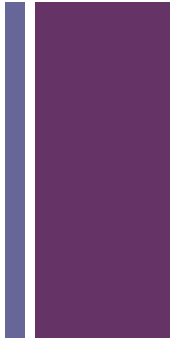
## ■ Phân xử bus (cũng sử dụng vector ngắt)

- Sử dụng cơ chế cho phép một module I/O chiếm quyền sử dụng bus rồi mới gửi yêu cầu ngắt
- Khi bộ xử lý phát hiện ra ngắt, nó trả lời trên đường ACK.
- Module I/O đặt vector ngắt của nó lên các đường dữ liệu
- VXL sử dụng vector này trở tới trình phục vụ ngắt tương ứng của TB





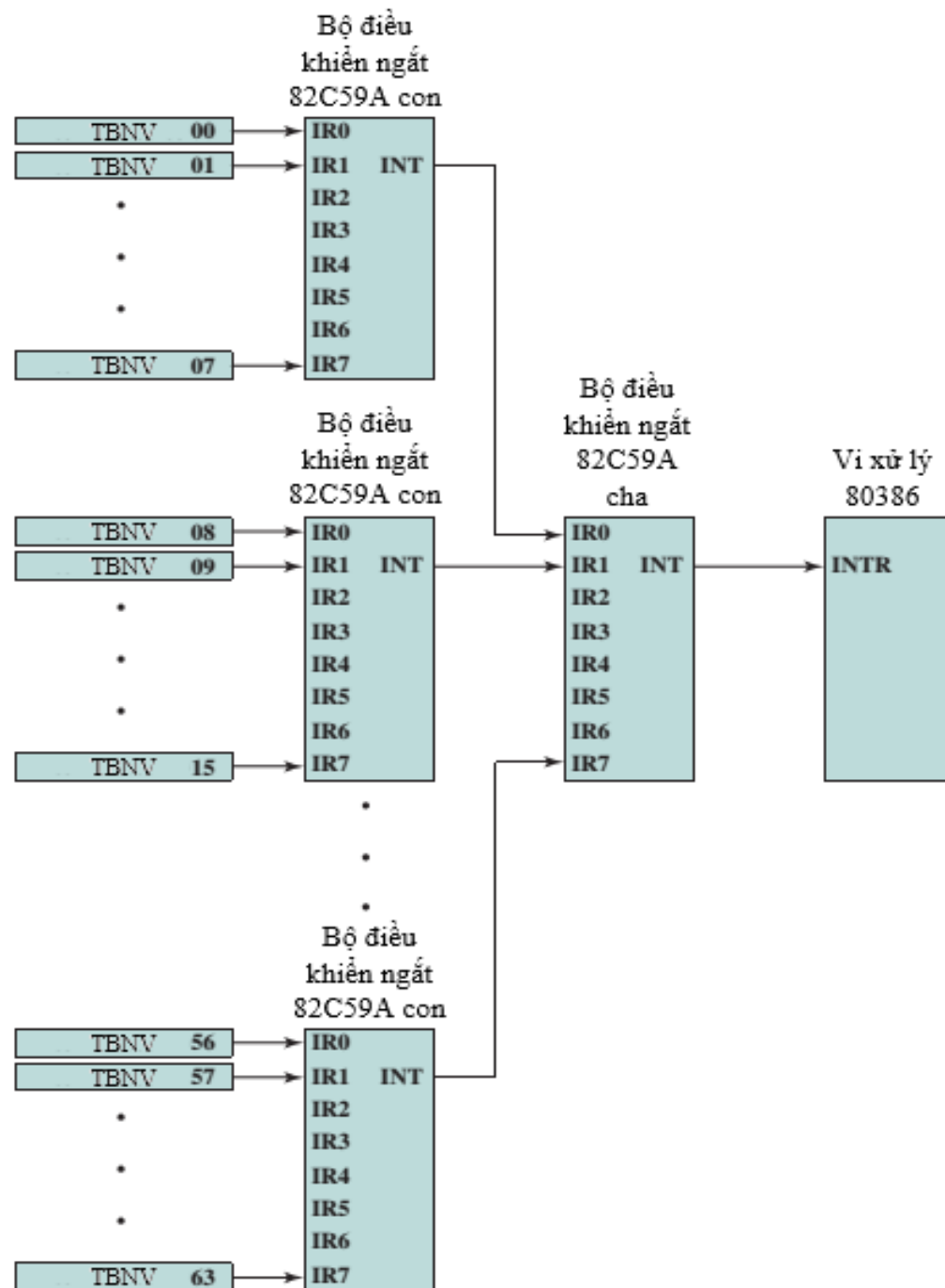
## 2. Xác định ưu tiên



- Các phương pháp nhận diện thiết bị đồng thời cho phép xác định độ ưu tiên của các TB khi có nhiều yêu cầu ngắt cũng một thời điểm:
  - Nhiều đường ngắt: VXL sẽ chọn đường có độ ưu tiên cao hơn để xử lý trước
  - Thăm dò phần mềm: thứ tự thăm dò các thiết bị được sắp xếp theo độ ưu tiên
  - Chuỗi daisy: tương tự thăm dò phần mềm
  - Phân xử bus: cơ chế phân xử bus đã có phân xử theo độ ưu tiên

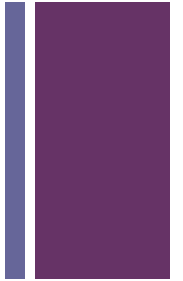
# + Ví dụ

- Kiến trúc dòng máy sử dụng chip Intel 80386: thực hiện việc điều khiển ngắt thông qua vi điều khiển Intel 82C59A





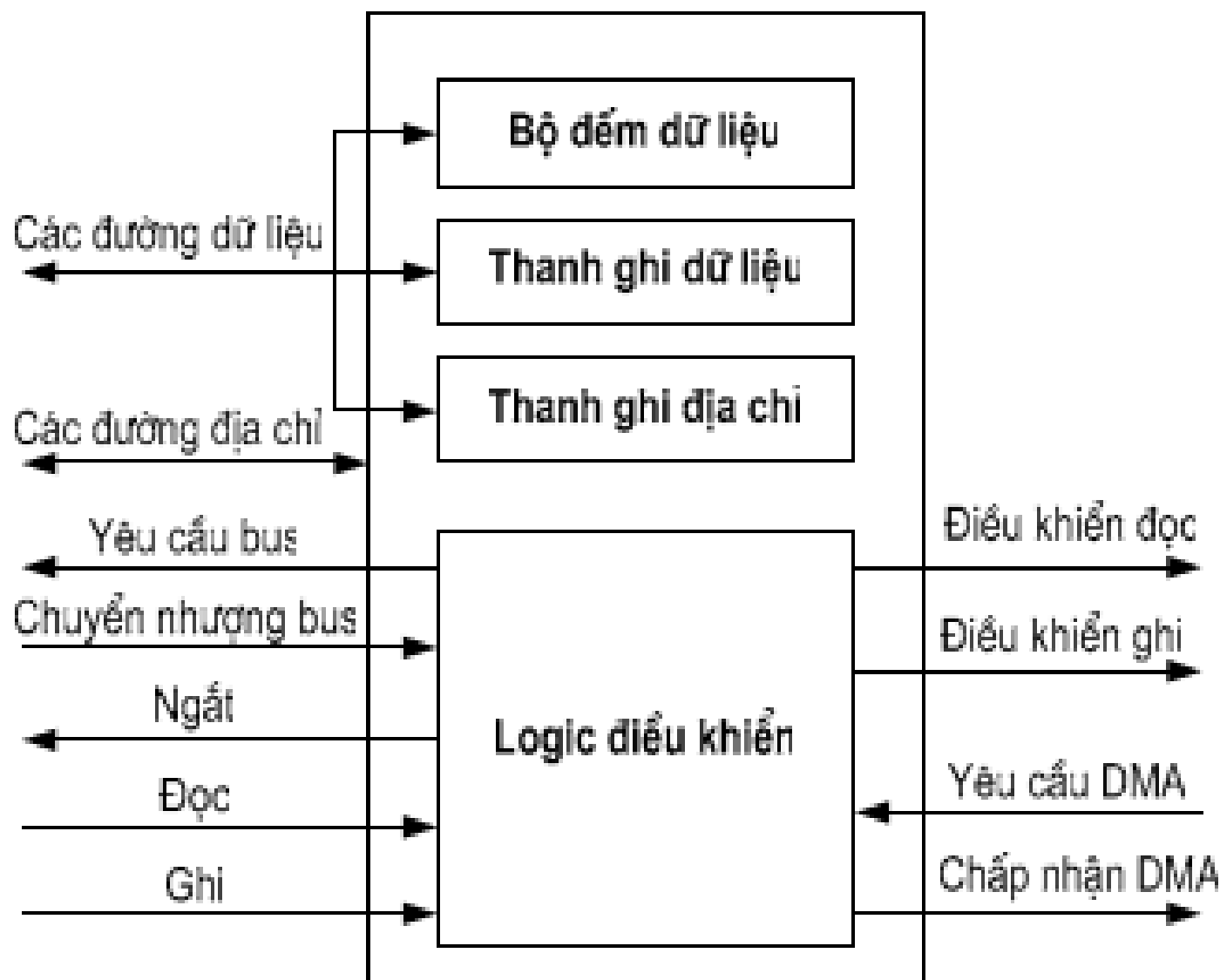
## c. Truy cập bộ nhớ trực tiếp - DMA



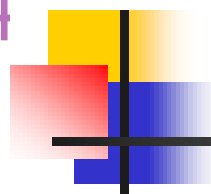
- Nhược điểm của I/O chương trình và I/O điều khiển ngắt: VXL thay gia vào hầu hết chu trình truyền/nhận dữ liệu giữa TBNV và BN
  - 1) Tốc độ truyền I/O bị giới hạn bởi tốc độ kiểm tra và phục vụ thiết bị của bộ xử lý
  - 2) Bộ xử lý gắn với việc quản lý truyền I/O; Một số chỉ lệnh phải được thực hiện cho mỗi lần truyền I/O

**Khi khối lượng dữ liệu lớn được di chuyển, một kỹ thuật hiệu quả hơn là truy cập bộ nhớ trực tiếp (DMA)**

# Sơ đồ cấu trúc của DMAC



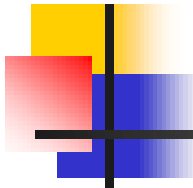




## Các thành phần của DMAC

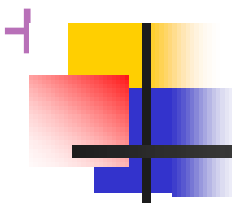
---

- Thanh ghi dữ liệu: chứa dữ liệu trao đổi
- Thanh ghi địa chỉ: chứa địa chỉ ngăn nhớ dữ liệu
- Bộ đếm dữ liệu: chứa số từ dữ liệu cần trao đổi
- Logic điều khiển: điều khiển hoạt động của DMAC



# Hoạt động DMA

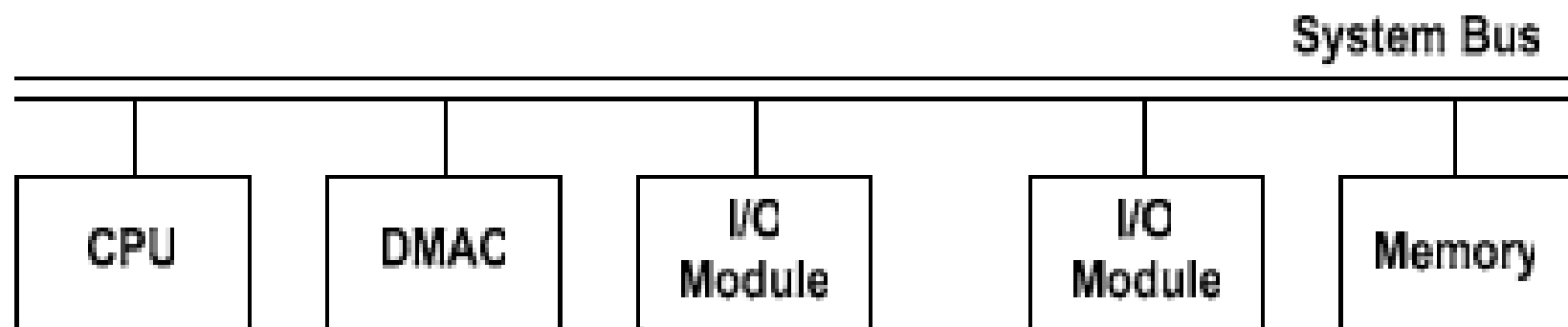
- CPU “nói” cho DMAC
  - Vào hay Ra dữ liệu
  - Địa chỉ thiết bị vào-ra (cổng vào-ra tương ứng)
  - Địa chỉ đầu của mảng nhớ chứa dữ liệu → nạp vào thanh ghi địa chỉ
  - Số từ dữ liệu cần truyền → nạp vào bộ đếm dữ liệu
- CPU làm việc khác
- DMAC điều khiển trao đổi dữ liệu
- Sau khi truyền được một từ dữ liệu thì:
  - nội dung thanh ghi địa chỉ tăng
  - nội dung bộ đếm dữ liệu giảm
- Khi bộ đếm dữ liệu = 0, DMAC gửi tín hiệu ngắt CPU để báo kết thúc DMA



## Các kiểu thực hiện DMA

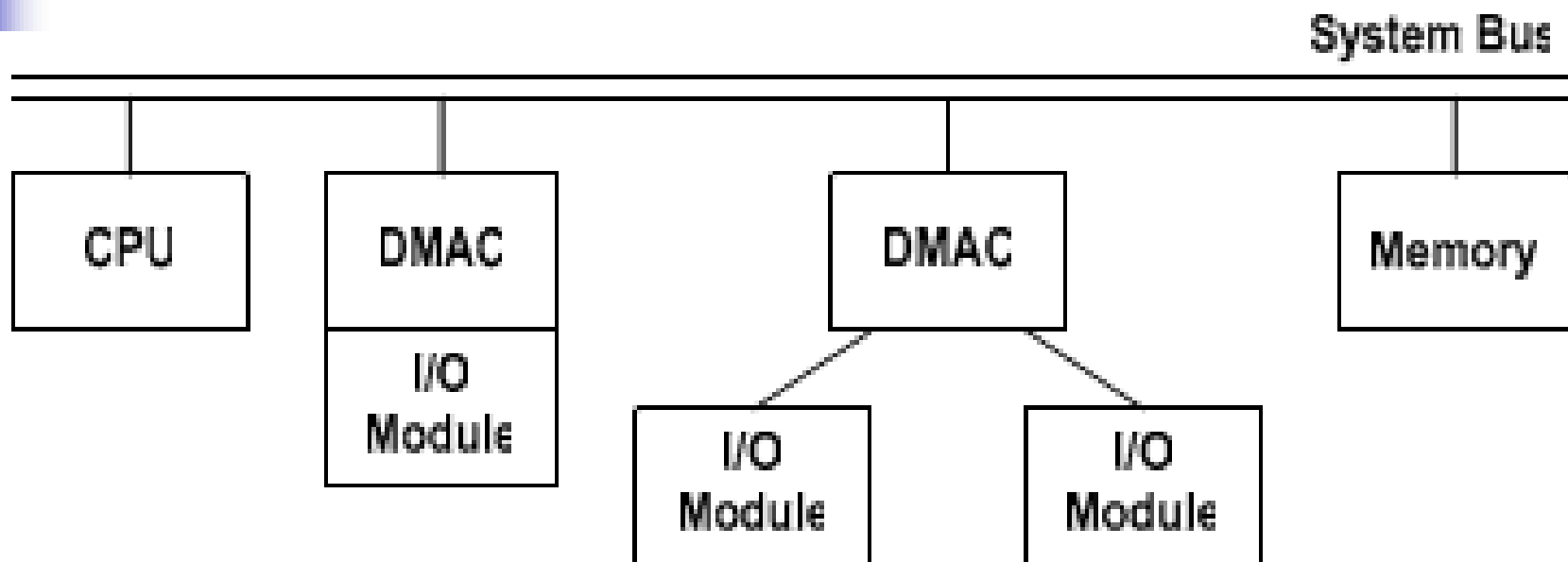
- DMA truyền theo khối (Block-transfer DMA): DMAC sử dụng bus để truyền xong cả khối dữ liệu
- DMA lấy chu kỳ (Cycle Stealing DMA): DMAC cưỡng bức CPU treo tạm thời từng chu kỳ bus, DMAC chiếm bus thực hiện truyền một từ dữ liệu.
- DMA trong suốt (Transparent DMA): DMAC nhận biết những chu kỳ nào CPU không sử dụng bus thì chiếm bus để trao đổi một từ dữ liệu.

# Cấu hình DMA (1)

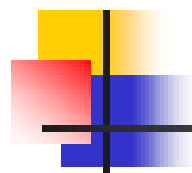


- Mỗi lần truyền, DMAC sử dụng bus hai lần
  - Giữa mô-đun vào-ra với DMAC
  - Giữa DMAC với bộ nhớ

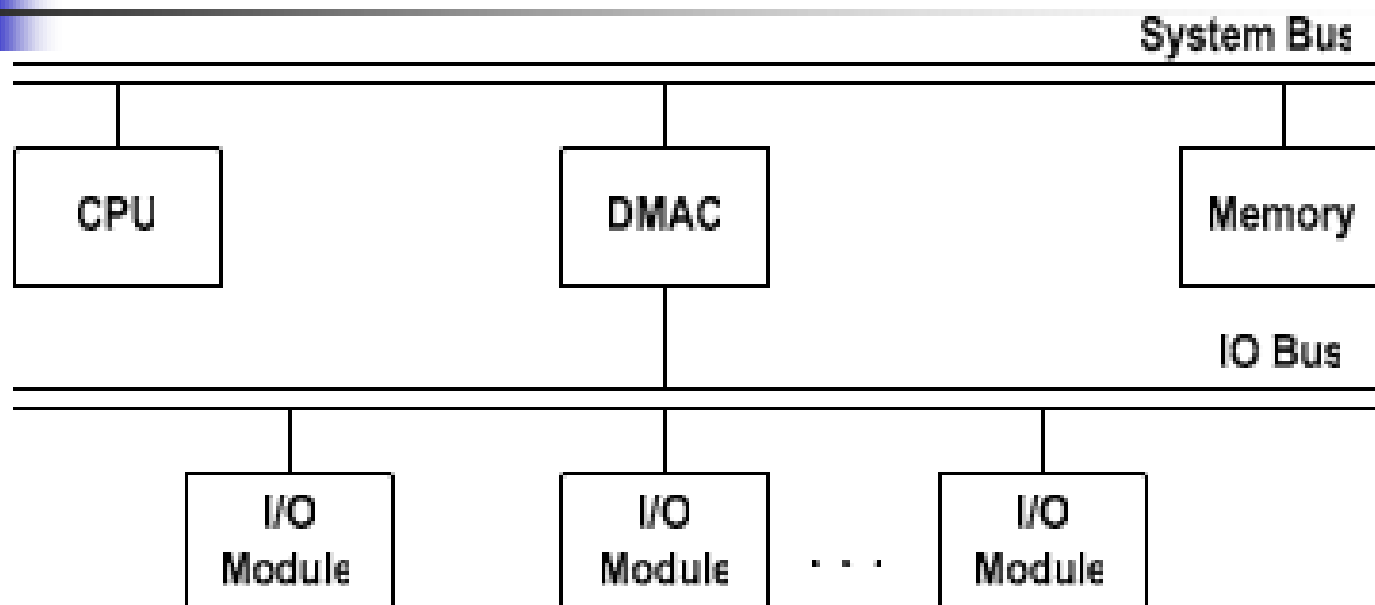
## Cấu hình DMA (2)



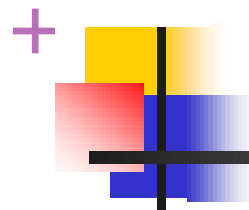
- DMAC điều khiển một hoặc vài mô-đun vào-ra
- Mỗi lần truyền, DMAC sử dụng bus một lần
  - Giữa DMAC với bộ nhớ



## Cấu hình DMA (3)



- Bus vào-ra tách rời hỗ trợ tất cả các thiết bị cho phép DMA
- Mỗi lần truyền, DMAC sử dụng bus một lần
  - Giữa DMAC với bộ nhớ



## Đặc điểm của DMA

---

- CPU không tham gia trong quá trình trao đổi dữ liệu
- DMAC điều khiển trao đổi dữ liệu giữa bộ nhớ chính với mô-đun vào-ra (hoàn toàn bằng phần cứng) → tốc độ nhanh
- Phù hợp với các yêu cầu trao đổi mảng dữ liệu có kích thước lớn

# + Chức năng DMA



- DMA bao gồm một module bổ sung trên hệ thống bus.
- Module DMA có khả năng thực hiện việc điều khiển việc truyền/nhận dữ liệu thay cho VXL
- Module DMA chỉ sử dụng bus khi VXL không cần đến nó hoặc buộc VXL phải tạm ngừng hoạt động để chiếm bus. Kỹ thuật thứ hai là phổ biến hơn.

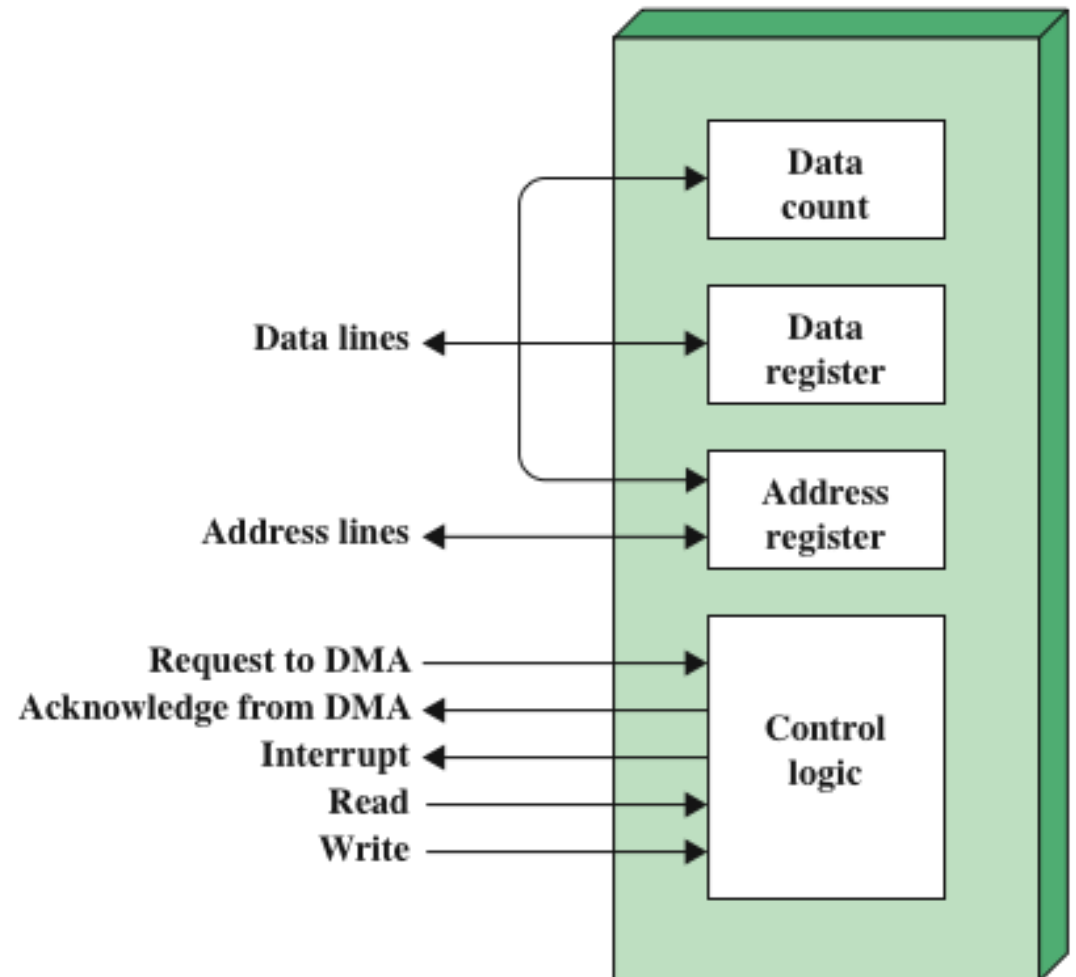
## **Quá trình đọc/ghi dữ liệu sử dụng DMA**

- Khi cần đọc/ghi dữ liệu, VXL gửi đến module DMA các thông tin sau:
    - Yêu cầu đọc/ghi: đường điều khiển
    - Địa chỉ TBNV: đường dữ liệu
    - Vị trí bộ nhớ bắt đầu để lưu trữ dữ liệu: đường dữ liệu và lưu trữ trên thanh ghi địa chỉ
    - Số lượng word được đọc/ghi: đường dữ liệu, lưu trữ trên thanh ghi data count
  - VXL thực hiện công việc khác, DMA thực hiện việc truyền giữa BN và TBNV
  - Sau khi hoàn thành, DMA gửi tín hiệu ngắt cho VXL
- VXL chỉ tham gia vào thời điểm bắt đầu và kết thúc việc truyền tin





## Sơ đồ module DMA điển hình

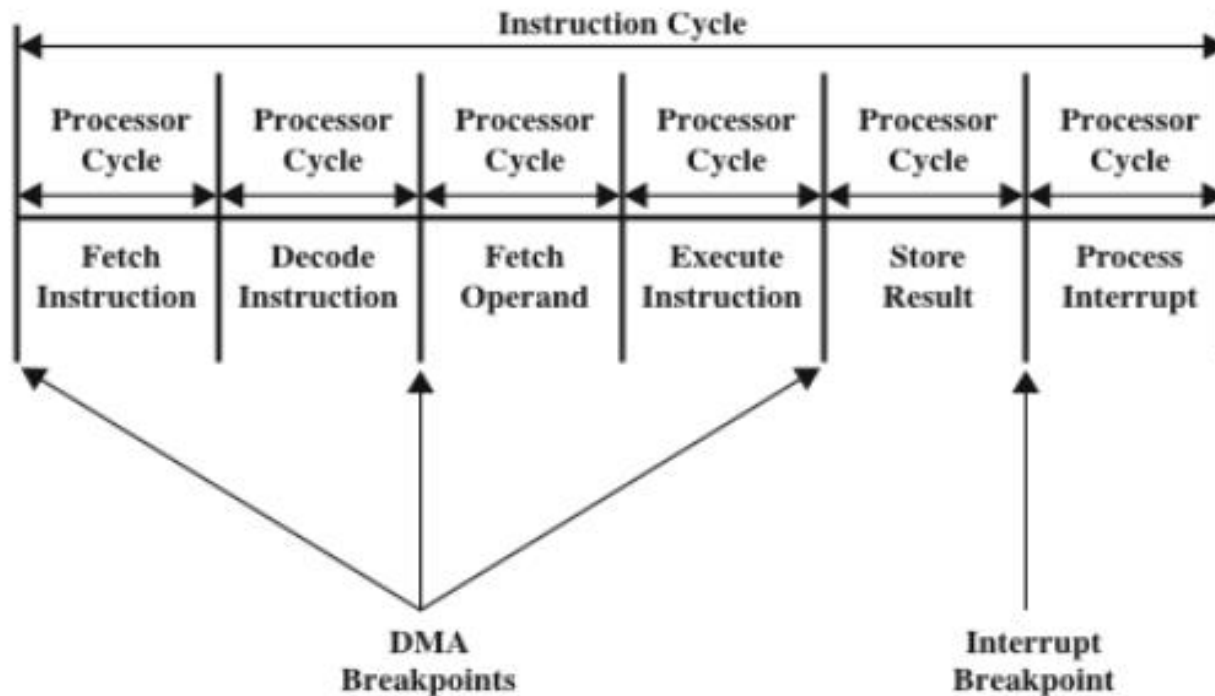




## Chu kỳ DMA

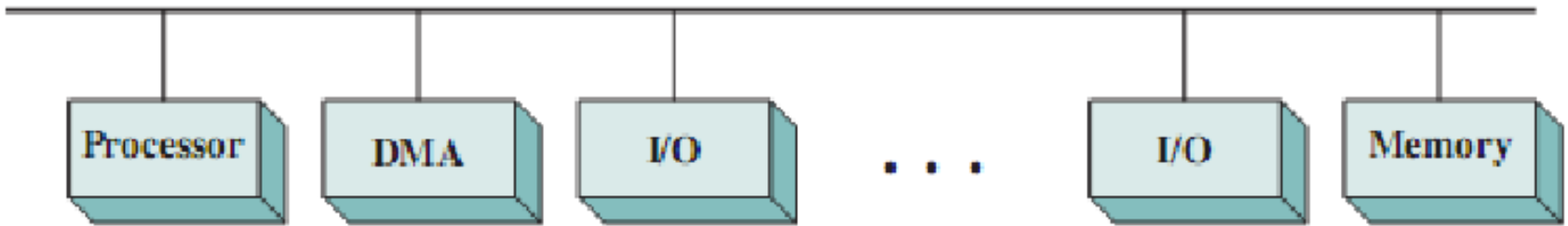
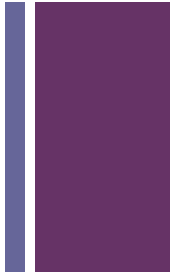


- Khi có yêu cầu bus từ DMA, VXL phải tạm thời “treo” để nhường bus cho DMA.
- Do không phải là ngắt nên VXL chỉ tạm dừng trong một chu kỳ bus
- DMA cũng làm cho VXL bị chậm hơn, tuy nhiên, khi truyền một lượng lớn dữ liệu thì DMA hiệu quả hơn nhiều so với 2 kỹ thuật còn lại

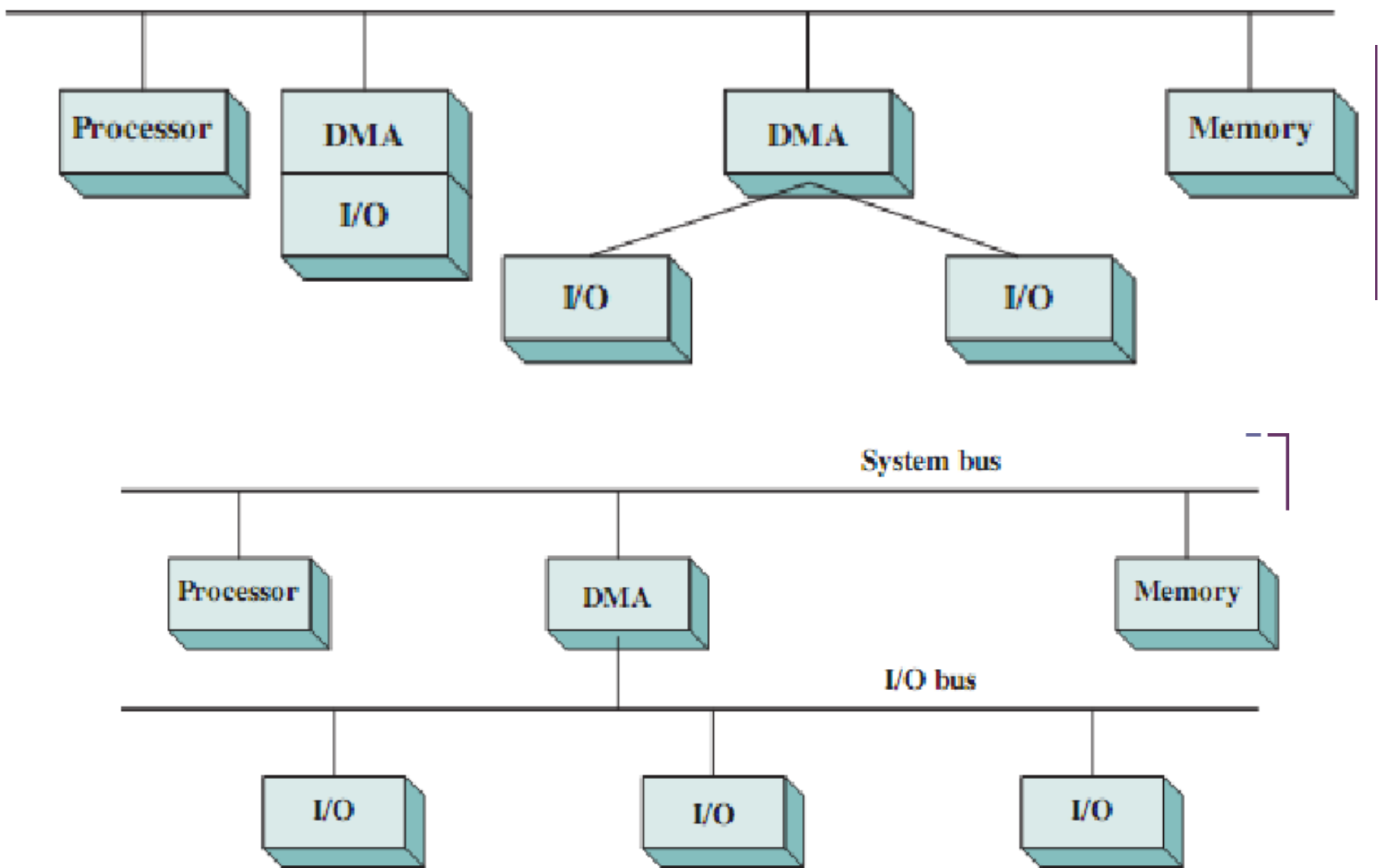




## c. Cấu hình DMA



- Cấu hình DMA: tất cả các module chia sẻ chung bus hệ thống
- Module DMA điều khiển việc truyền dữ liệu giữa I/O và memory trong hai chu kỳ bus
- Cấu hình có chi phí thấp nhưng rõ ràng không hiệu quả



- DMA giao tiếp với I/O không qua bus hệ thống: giảm được chu kỳ bus
- DMA chỉ chiếm bus hệ thống khi cần trao đổi dữ liệu với bộ nhớ chính



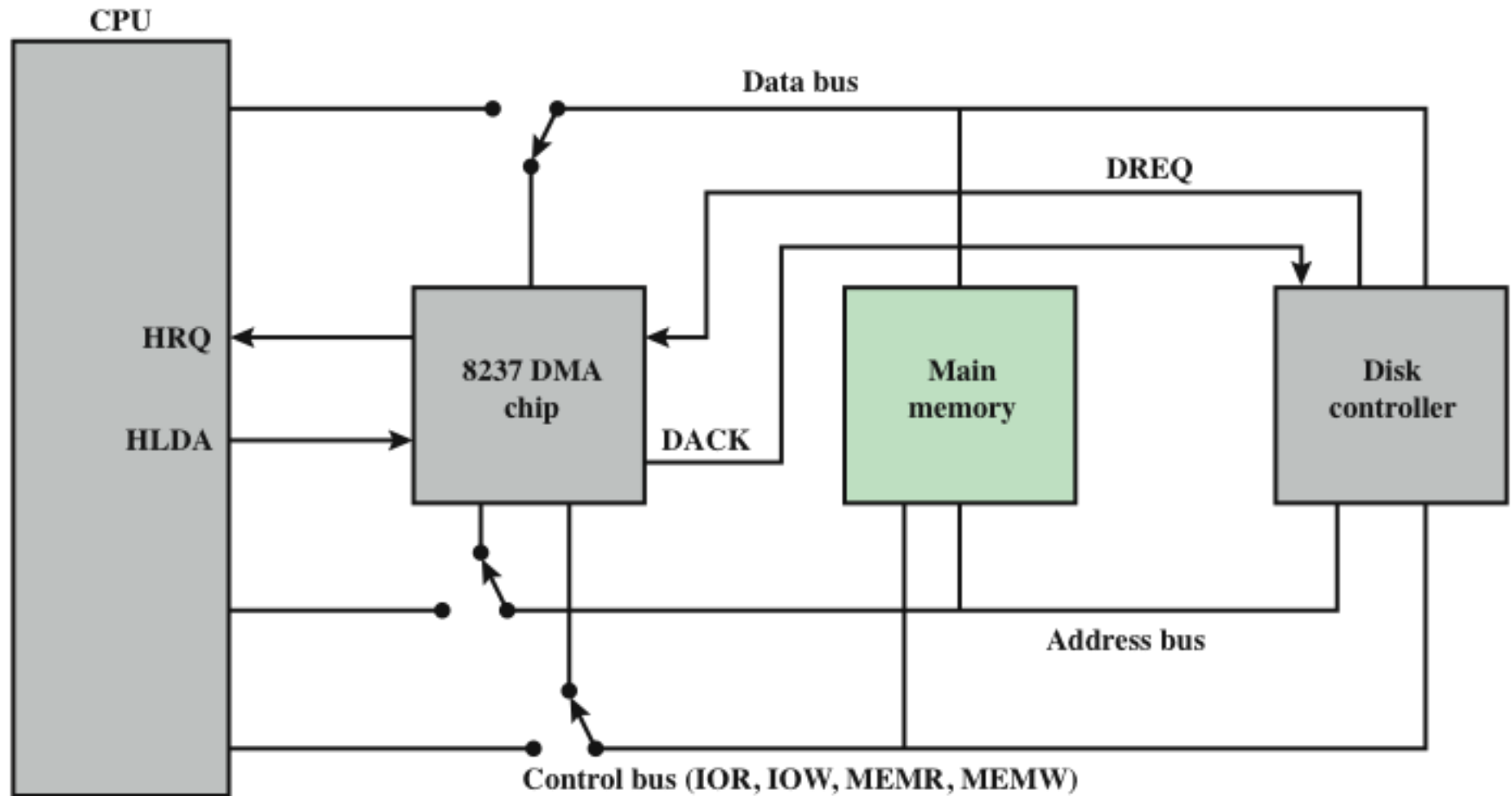
## d. Bộ điều khiển DMA Intel 8237



- Giao tiếp với họ 80x86 và DRAM
  - Khi module DMA cần bus nó sẽ gửi tín hiệu HOLD tới bộ vi xử lý
  - CPU phản hồi HLDA (hold acknowledge) - Mô-đun DMA có thể sử dụng bus
  - Ví dụ: truyền dữ liệu từ bộ nhớ tới đĩa
1. Thiết bị yêu cầu DMA bằng cách nâng DREQ (yêu cầu DMA)
  2. DMA đặt HRQ (yêu cầu giữ) lên cao
  3. CPU kết thúc chu kỳ bus hiện tại và đặt HDLA lên cao. HOLD duy trì trong suốt thời gian DMA
  4. DMA kích hoạt DACK (DMA ack), báo thiết bị bắt đầu truyền
  5. DMA bắt đầu truyền bằng cách đặt byte đầu tiên của địa chỉ lên bus địa chỉ và kích hoạt MEMR; sau đó kích hoạt IOW để ghi vào ngoại vi. DMA giảm bộ đếm và tăng con trỏ địa chỉ. Lặp lại cho đến khi đếm về 0
  6. DMA hủy HRQ, trả bus trở lại CPU

# 8237 DMA

## Cách sử dụng Bus Hệ thống



DACK = DMA acknowledge  
DREQ = DMA request  
HLDA = HOLD acknowledge  
HRQ = HOLD request



## d. Bộ điều khiển DMA Fly-By



- Khi DMA sử dụng bus, bộ xử lý nghỉ. Khi bộ xử lý sử dụng bus, DMA nghỉ. DMA 8237 được gọi là bộ điều khiển DMA fly-by
- Dữ liệu không đi qua, không lưu trữ trong chip DMA
  - DMA giữa cổng I/O và bộ nhớ
  - Không đặt giữa hai cổng I/O hoặc hai vị trí bộ nhớ
- Thực hiện giao tiếp bộ nhớ - bộ nhớ qua thanh ghi
- 8237 có bốn kênh DMA
  - Lập trình độc lập
  - Kênh nào cũng có thể hoạt động tại 1 thời điểm
  - Được đánh số 0, 1, 2, và 3

| Bit | Command                       | Status                   | Mode                                    | Single Mask             | All Mask                     |
|-----|-------------------------------|--------------------------|-----------------------------------------|-------------------------|------------------------------|
| D0  | Memory-to-memory E/D          | Channel 0 has reached TC | Channel select                          | Select channel mask bit | Clear/set channel 0 mask bit |
| D1  | Channel 0 address hold E/D    | Channel 1 has reached TC |                                         |                         | Clear/set channel 1 mask bit |
| D2  | Controller E/D                | Channel 2 has reached TC |                                         | Clear/set mask bit      | Clear/set channel 2 mask bit |
| D3  | Normal/compressed timing      | Channel 3 has reached TC | Verify/write/read transfer              | Not used                | Clear/set channel 3 mask bit |
| D4  | Fixed/rotating priority       | Channel 0 request        | Auto-initialization E/D                 |                         | Not used                     |
| D5  | Late/extended write selection | Channel 0 request        | Address increment/decrement select      |                         |                              |
| D6  | DREQ sense active high/low    | Channel 0 request        |                                         |                         |                              |
| D7  | DACK sense active high/low    | Channel 0 request        | Demand/single/block/cascade mode select |                         |                              |



Bảng 7.2  
Thanh ghi  
Intel  
8237A

E/D = enable/disable                      TC = terminal count



## + 6. VXL và các kênh I/O



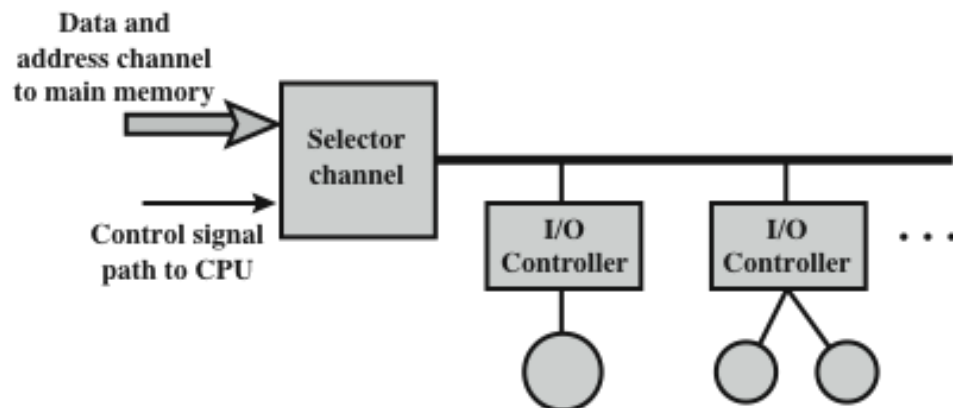
### a. Sự phát triển của chức năng I/O qua các thời kỳ

1. CPU trực tiếp điều khiển một thiết bị ngoại vi.
2. Thêm vào Một bộ điều khiển hoặc module I/O. CPU sử dụng I/O lập trình, không có ngắt.
3. Cấu hình tương tự như bước 2, nhưng có sử dụng ngắt. CPU không phải tốn nhiều thời gian chờ đợi một hoạt động I/O được thực hiện, do đó tăng hiệu quả.
4. Module I/O được truy cập trực tiếp tới bộ nhớ qua DMA. Nó có thể di chuyển một khối dữ liệu đến/từ bộ nhớ mà không liên quan đến CPU, ngoại trừ khi bắt đầu và kết thúc quá trình truyền.
5. Module I/O được tăng cường để trở thành một bộ xử lý theo quyền riêng của nó, với một tập hợp chỉ lệnh dành riêng cho I/O
6. Module I/O có bộ nhớ cục bộ riêng và trên thực tế là một máy tính theo quyền riêng của nó. Với kiến trúc này, có thể kiểm soát một tập hợp lớn các thiết bị I/O với sự tham gia tối thiểu của CPU.

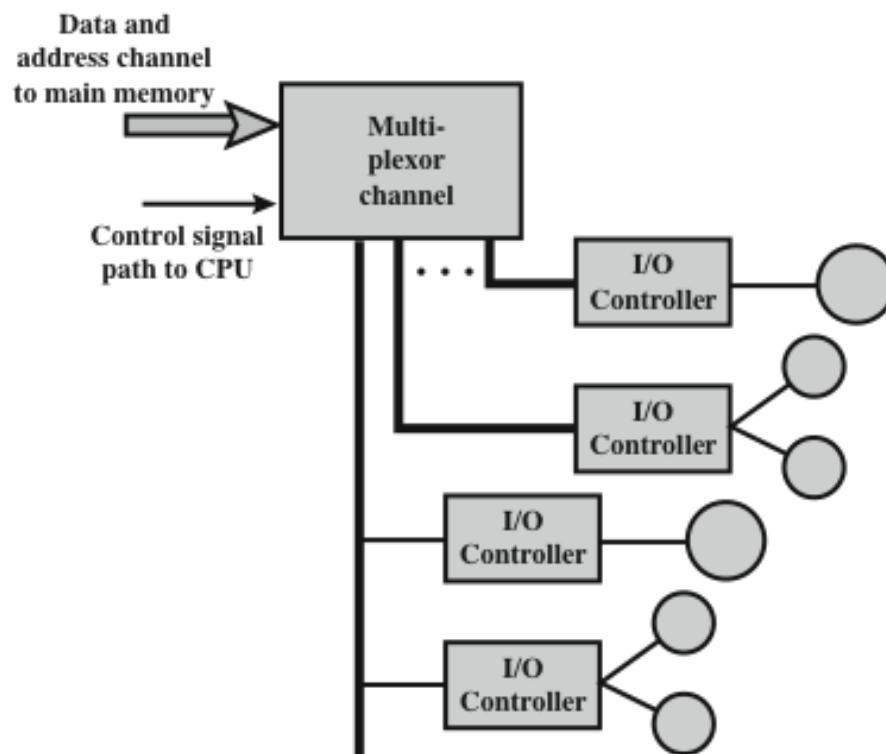
I/O channel



# Kiến trúc I/O channel

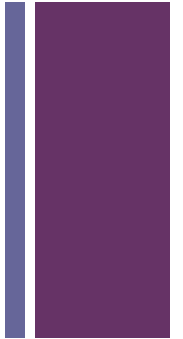


(a) Selector



(b) Multiplexor

# + Review Questions



1. Liệt kê ba loại thiết bị ngoại vi (thiết bị ngoài).
2. IRA - International Reference Alphabet là gì?
3. Các chức năng chính của module I/O là gì?
4. Trình bày ba kỹ thuật để thực hiện I/O.
5. Sự khác nhau giữa I/O ánh xạ bộ nhớ và I/O riêng biệt là gì?
6. Khi một ngắt được gửi đến VXL, bộ xử lý xác định thiết bị đã yêu cầu ngắt như thế nào?
7. Trong khi một module DMA chiếm quyền điều khiển bus VXL làm gì?